

Retrieval-Augmented Generation (RAG):

Retrieval-Augmented Generation or RAG is a way of building a question answering system where instead of asking a language model to answer purely from what it learned during training, the system first searches through a set of documents for relevant passages and then hands those passages to the model along with the question. The core idea is similar to what I already did with the text classification task, we still have an input and we still want a useful output, except now the "input" the model reasons over is dynamically pulled from a document collection instead of being taken into the model's weights ahead of time.

This shift matters because it changes what the model is actually doing. Instead of recalling an answer from memory, it is reading evidence and answering based on it which is a much easier and more reliable thing for it to do well.

Why Plain LLMs Hallucinate:

Language models are trained to predict a plausible next chunk of text, not to check facts against a database. When a model is asked about something it did not see enough of during training or something that changed after its training cutoff or something private like a company's internal documents, it does not have a built in way to say "I don't know." It just produces the most statistically likely sounding answer, said with full confidence whether or not it is actually correct. This is what people mean by hallucination.

This gets worse the more specific or private the material is. A model has never seen our personal files, our company's internal policies or a niche research paper that only exists on our computer, so asked about these directly it has nothing real to draw from and just guesses in a convincing tone. RAG deals with this by grounding every answer in text that was actually retrieved so the model is reasoning over real content instead of pattern matching from general training data.

Document Loading:

The first practical step in building this kind of system is getting the source documents into a usable format. Documents show up as PDFs, Word files, web pages, plain text, sometimes even scanned images and each format needs its own extraction approach. A document loader's job is to pull out clean readable text and throw away the noise around it, things like page numbers, running headers, broken line wraps and leftover encoding artifacts.

A good loader also keeps track of metadata such as the file name, page number or section heading, since this is what lets the final system later tell a user exactly which document an answer came from instead of just returning a wall of text with no source attached.

Text Chunking:

Documents are almost always too long to search as one single block. A fifty-page PDF might only have one paragraph that actually answers a given question so chunking splits documents into smaller overlapping pieces, usually a few hundred words at a time so later steps can pull out just the relevant piece instead of the whole file.

Picking chunk size is a bit of a balancing act. Chunks that are too small lose surrounding context and can cut a sentence's meaning in half while chunks that are too large end up diluting relevance

since the right answer might be buried among a lot of unrelated text. Overlapping consecutive chunks slightly, so the end of one repeats at the start of the next, helps avoid losing information that happens to fall right on a chunk boundary.

Embedding's:

An embedding is just a numerical representation of a piece of text, a list of a few hundred to a few thousand numbers that captures its meaning in mathematical form. Text with a similar meaning ends up with embedding's that sit close together in that numerical space, even when the actual words used are completely different.

This is what lets a system match a question like "how do I reset my password" with a chunk that says "follow these steps to recover account access" even though the two sentences barely share any words. Embedding's come from models trained specifically to capture semantic meaning rather than literal word overlap which is what makes this kind of matching possible in the first place.

Vector Search:

Once every chunk from every document has been turned into an embedding, those embeddings get stored in a vector database which is built specifically to store and search through large numbers of these numerical vectors quickly. A vector database can hold millions of embeddings and still return the closest matches to a new query in a fraction of a second using specialized indexing rather than comparing against every single stored item one by one.

When someone asks a question, that question gets converted into an embedding using the same embedding model that was used on the documents and the vector database finds which stored chunks sit closest to that question's embedding.

Similarity Search:

Similarity search is the actual method used to measure how close two embeddings are to each other. The most common approach is cosine similarity which looks at the angle between two vectors rather than their raw distance which ends up being pretty effective at capturing meaning regardless of how long or short the text was to begin with. Other measures like Euclidean distance or dot product get used to depending on the embedding model and vector database being used.

What comes out of a similarity search is a ranked list, the chunks whose embeddings are mathematically closest to the question, ordered from most to least relevant.

Retriever:

The retriever is what ties embeddings and similarity search together into one reusable step. Given a question, it handles embedding the query, searching the vector database and returning the top matching chunks usually somewhere between three and ten passages depending on how the system is set up.

Retrievers can get more sophisticated than a plain similarity lookup too. Some combine keyword search together with vector search which is usually called hybrid search to catch cases where an

exact term matters a lot, like product codes or names which pure semantic similarity sometimes misses entirely.

Context Injection:

Once the retriever finds the most relevant chunks, those chunks need to actually get inserted into the prompt sent to the language model along with the original question, which is what context injection means. The prompt is usually written so the model is explicitly told to answer using only the passages it was given, and to say so if those passages don't actually contain enough information to answer confidently.

This is really the point where retrieval and generation meet. The model never sees the whole document collection directly, only the small number of passages the retriever picked out for it, which is what keeps its answer grounded and keeps the amount of text it has to work through manageable.

Source-Based Answering:

The last piece is showing the answer back to the user along with where it came from. Since every retrieved chunk carries metadata from the document loading step, the system can point to exactly which document, and often which page or section, backed up each part of the answer. This is what turns the whole thing from a black box into something a person can actually go check for themselves.

This is really what builds trust in these systems more than anything else. Someone asking a question about a two-hundred-page policy document does not just want an answer, they want to know they can verify it. Being able to trace a generated answer back to the exact passage it came from is what separates a genuinely useful document QA system from something that just sounds confident.

Conclusion:

The full pipeline ends up looking like this: documents get loaded and cleaned, split into overlapping chunks, and converted into embeddings that get stored in a vector database. When someone asks a question, that question gets embedded the same way, compared against the stored chunks using similarity search, and the most relevant passages get pulled out. Those passages get injected into a prompt along with the question and the model generates an answer grounded in that context with the sources cited right alongside it.

Each stage is solving one specific weakness of using a language model by itself loading and chunking make large document collections searchable at all, embeddings and vector search make that searching based on meaning instead of exact keywords, the retriever ties the search into the actual pipeline and context injection combined with citing sources is what keeps the final answer honest and something a person can check.

References:

<https://www.geeksforgeeks.org/nlp/what-is-retrieval-augmented-generation-rag/>

<https://www.pinecone.io/learn/retrieval-augmented-generation/>

<https://gpt-trainer.com/blog/rag+chunking+strategy>